A Bit-Exact, Processor-Free FPGA Adaptive Noise Canceller: Fair Comparison of LMS, NLMS and Divider-Free NLMS-Shift on Cyclone IV E

Satvik Garg & Aditya Gupta & Jaffino G[a,*]

[a]*School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore, India*

Abstract

Adaptive noise cancellation with a reference microphone is a classical application of the least-mean-square (LMS) family, and a small field-programmable gate array (FPGA) can perform it at audio rates without a processor. In this paper, ULLAP, a 12-bit, 32-tap noise canceller, is presented for the DE2-115 board (Cyclone IV E) with a WM8731 codec. A single serial core with one shared multiplier implements LMS, normalized LMS (NLMS) with a restoring divider, and NLMS-Shift, in which the division is replaced by a leading-one detector and a shift. With the step sizes tuned on held-out seeds, NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise (22.37 and 24.33 dB over 20 seeds) and outperforms LMS by about 2 dB on amplitude steps, while needing 72 instead of 130 busy cycles per sample. Moreover, NLMS-Shift is shown to be equal to NLMS with a level-dependent step between 0.5 and 1 times the nominal step. On real recorded noise and speech, the filters remove 11.7–14.9 dB and raise the short-time objective intelligibility from 0.69–0.78 to 0.95–0.98, whereas without speech they remove 18.5–31.9 dB, which shows that the speech, and not the noise, limits the performance. A 12-bit word keeps the quantization loss below 0.4 dB on broadband noise. On the board, all 724,459 captured output samples of the three algorithms are equal to those of the integer model, and the measured sample rate, latency and busy cycles match the register-transfer-level design.

*Corresponding author. [TBD: e-mail and author order to be confirmed by the authors]

Keywords: adaptive noise cancellation, NLMS, divider-free normalisation, FPGA, fixed-point arithmetic, bit-exact verification

1. Introduction

Noise is present in almost every place where people live and work, such as the hum of an office air conditioner, a compressor near a production line, road noise inside a vehicle cabin and the background chatter around a hearing-aid user. When a second microphone can be placed close to the noise source, the noise that reaches the main microphone can be predicted from this reference and subtracted from it. The predictor used for this purpose is an adaptive filter, which learns the acoustic path between the two microphones during operation and tracks the path whenever it changes. The least-mean-square (LMS) algorithm and its normalized form (NLMS) are still the most widely used algorithms for this filter, since they need only a few multiply–accumulate operations per tap and per sample, and their behavior has been analyzed for several decades [1].

In a small embedded device, this filter must operate in real time with a short and fixed delay and with very low energy consumption. A field-programmable gate array (FPGA) is well suited to this requirement, because the datapath is built around the algorithm, the latency is known to the exact clock cycle, and no processor or operating system is required for scheduling. However, LMS and NLMS are not equally simple to realize in hardware. LMS needs only multipliers and adders, but its convergence speed depends on the level of the input signal. NLMS removes this dependence by dividing the step size by the input power, which requires one division in every sample. Since dividers are slow and occupy a large amount of logic, recent designs approximate the division instead, using an approximate Newton–Raphson iteration [2], canonical-signed-digit reciprocals [3], or a binary reciprocal formed from a leading-one detector and a shifter [4]. The simplest option rounds the input power to a power of two, so that the division reduces to a plain shift. In this work, this variant is referred to as NLMS-Shift.

Two issues in the existing literature motivate this work. The first issue is tuning. The compared variants are rarely tuned with the same care, and hence a reported gain may come from a weak baseline rather than from the algorithm itself. An early version of this work suffered from the same problem: NLMS-Shift appeared 5.7 dB better than NLMS, and the gap

disappeared once NLMS was given a suitable regularization constant. The second issue is the link between the model and the silicon. Hardware papers generally report area, clock rate and power. These figures prove that a design fits on the chip, but they do not prove that the chip produces the same samples as the model from which the accuracy results were obtained.

In this paper, ULLAP, a 12-bit, 32-tap adaptive noise canceller, is presented for the low-cost Terasic DE2-115 board (Intel Cyclone IV E) with its WM8731 audio codec, and it is used to address both issues. The major contributions of this work are given below.

1. **Single serial core for three algorithms:** A single serial core is designed to run LMS, NLMS (with a restoring divider) and NLMS-Shift on one shared multiplier, so that the three algorithms share the same datapath, word length and filter length. The core operates inside a complete processor-free audio system, closes timing at 50 MHz on Cyclone IV E, and is released along with all scripts and data.
2. **Fair comparison:** The step size and the regularizer of each algorithm are selected on held-out seeds using the same rule, and every result is reported with a 20-seed 95% confidence interval. The comparison covers synthetic scenarios as well as real recorded noise and speech, and includes a reproducible word-length study.
3. **Bit-exact verification chain:** A bit-exact verification chain is built from a Python integer model through the RTL to the board wrapper (86 regression tests, including codec, freeze and reset behavior). On the board, captures of real codec samples show that every output sample of all three algorithms is equal to that of the model.
4. **Two measured explanations:** The first explanation concerns the shift. NLMS-Shift is shown to be NLMS with a step that varies between 0.5 and 1 times the nominal step as the input level changes. NLMS operated with the average of this step reproduces NLMS-Shift within 0.3 dB on broadband noise, whereas on tonal input the step jumps whenever the window power crosses a power of two. The second explanation concerns the real recordings. When the speech is removed, the same filters remove 18.5–31.9 dB of the same real noise. Hence, the lower noise reduction obtained with speech is caused by the speech disturbing the adaptation and not by the noise.

The shift normalizer itself is not new and is not claimed as a contribution; Section 2 places it among the earlier division-free designs.

The rest of the paper is organized as follows. Section 2 reviews the problem and the related works. Section 3 gives an overview of the system, Section 4 describes the algorithms and the fixed-point model, and Section 5 explains the hardware architecture. Sections 6 and 7 present the verification and the experimental set-up, and Section 8 discusses the results. Finally, Sections 9 and 10 give the discussion and the concluding remarks.

2. Literature review

2.1. The problem: noise in daily life

At present, noise is regarded as an environmental health risk and not merely as a nuisance. In Europe, more than 113 million people are exposed to transport noise above 55 dB(A), and traffic noise alone is estimated to cause the loss of about 1.3 million healthy life-years every year through its effect on the heart and blood vessels [5]. Hahad et al. [6] reviewed the evidence that long-term noise exposure is also associated with mental-health disorders and discussed the biological mechanisms that may explain this association. Hearing loss forms the other side of the problem. According to the World Health Organization, about 1.5 billion people live with some degree of hearing loss, and this number is expected to reach 2.5 billion by 2050 [7]. For these listeners, background noise is the main reason why speech remains difficult to follow, even with a hearing aid. Reducing the noise at the microphone or near the ear is therefore useful in many settings, such as beside roads, where urban traffic noise lies mostly below 2 kHz [8], inside vehicle cabins [9, 10], in ventilation ducts and air conditioners [11], and in hearing aids [12, 13, 14]. Many of these devices are small, inexpensive and battery-powered, and hence the processing must run in real time at low cost and low energy.

2.2. Applications of adaptive noise control

Yang et al. [8] recorded road traffic noise at five sites in Beijing and developed a feedforward LMS prototype, which removed close to 30 dB(A) of single- and dual-frequency noise below 2 kHz. Jiang et al. [9] presented an in-car FxLMS system in which the step size depends on the reference norm, and the two parameters of this rule were tuned by particle-swarm optimization. On a dSPACE board, this system achieved up to 12 dB below 100 Hz and about 3 dB over 20–400 Hz in road tests, whereas plain FxLMS obtained between -0.8 and 0.3 dB. Zhu et al. [10] used the vehicle's own loudspeakers to control the first four engine orders. This system attained an

average reduction of 6.2 dB at 52 ear-level microphones, and in a jury test 93% of the listeners felt more comfortable with the system switched on. For hearing aids, Yin and Chen [12] introduced a weighted error into a variable-step adaptive feedback canceller, which converged 35% faster than a PEM-IMLMS baseline and 60% faster than PEM-NLMS on a real device. Husstedt et al. [13] evaluated six commercial behind-the-ear devices and observed that impulse noise reduction lowers the C-weighted peak levels by up to 20 dB, or by about 9 dB on average. A recent systematic review of speech enhancement for hearing aids [14] reported that deep networks perform best in complex acoustic scenes, while hybrid methods that retain an adaptive filter provide a better balance between latency and computation on wearable devices.

2.3. Adaptive algorithms

Lu et al. surveyed a decade of active noise control algorithms in two parts, the first on linear systems [1] and the second on nonlinear systems [15]. In both parts, LMS-type updates remain the base on which most of the newer methods are built. A popular approach is to run two filters at the same time, one fast and one accurate, and to combine their outputs. Félix et al. [16] proposed a modified convex combination of FXLMS filters, which converged faster and attenuated more than FXLMS and the earlier convex scheme. Zhu et al. [11] combined an adaptive FxLMS filter and a CNN–GRU network through a convex weight for cancelling harmonic noise in a bent air-conditioner pipe. A different problem occurs when the loudspeaker output leaks back into the reference microphone and the loop becomes unstable. Li et al. [17] addressed this problem with a Youla-parameterized feedforward controller and an adaptive IIR update, which was tested on a duct. All these methods improve the cancellation, but they add extra operations, networks or complete filters to every sample, which is contrary to the requirements of a small FPGA design.

2.4. Hardware realizations

In terms of hardware, Thannoon and Hashim [18] combined a fast retimed RLS filter and a slow LMS filter on an FPGA to remove power-line interference from ECG records of the MIT-BIH database. This design achieved 24.2% higher SNR and 50% faster convergence than conventional structures. Jagadeeswari et al. [19] applied cut-set retiming to delayed NLMS filters for audio on a Spartan-6 device and reduced the critical path from 29.96 to 13.39 ns. For underwater sonar, Karakaya and Aytar [20] implemented

semi-parallel 16-bit NLMS and variable-step LMS filters on an Artix-7 device, which raised the SNR from 8.85 dB to 29.36 and 28.57 dB using 2.51% and 2.91% of the LUTs, respectively. Park and Park [21] followed a different approach, in which a small CNN classifies the noise and selects one of several fixed filters; in simulation, this method obtained 2.3 dB more SNR than LMS with 53% less dynamic power. Abdi et al. [22] developed a narrowband controller around a variable-step LMS notch filter on an FPGA to suppress microphonics in a superconducting accelerator cavity. Other works focused on the arithmetic itself. Nagabushanam et al. [23] designed a multiplier-less LMS filter using distributed arithmetic with offset-binary coding, and da Rosa et al. [24] replaced the multipliers of an LMS power-line canceller with adders and shifters, which saved up to 4.98 times in power and 30.46% in area.

2.5. Removing the division from NLMS

The works closest to the present study are those that remove the NLMS division directly. Guidotti et al. [2] reduced the power of an NLMS interference canceller with an approximate Newton–Raphson integer divider, and Coelho et al. [25] lowered the cost of adaptive filtering using sum-of-powers-of-two arithmetic. Yao and Huang [3] approximated the reciprocal of the input power with canonical-signed-digit encoded divisors in a 16-tap NLMS FIR filter. Later, Huang and Yao [4] proposed a binary reciprocal approximation for an NLMS IIR filter. On the same DE2-115 board used in this work, their design consumed 5.07 nJ per sample against 8.33 nJ with a 15-bit SRT divider, and it reduced the energy per sample by 39.1%, 6.5%, 14.4% and 5.6% relative to the SRT, Newton–Raphson, CSD and lookup-table methods [4]. The same authors also noted that the older idea of rounding the divisor to a single power of two, which turns the division into a shift, has a large approximation error. This plain shift is exactly NLMS-Shift, and Section 8.4 shows that its error acts only as a level-dependent step factor between 0.5 and 1.

From the above review, no work was found that compares LMS, a true divider and a shift normalizer on one core with equal tuning and statistical tests, and only a few works test a complete audio system with a codec on a board. ULLAP adds this equal-effort three-way comparison, a bit-exact chain from the model to the hardware, and an evaluation on real recordings.

3. System overview

The overall system is shown in Fig. 1. The noise from the source v travels to the primary microphone through an unknown acoustic path h and is mixed there with the desired signal s. A reference microphone placed near the source picks up the noise v itself. The two microphones are connected to the left and right LINE IN channels of the WM8731 codec, which digitizes the signals and passes them to the FPGA over an I^{2}S link. Inside the FPGA, the I^{2}S receiver delivers one pair $(x[n], d[n])$ in every sample period to the ANC core. The core returns the error $e[n]$, which is the cleaned signal, and the I^{2}S transmitter sends this signal back to the codec DAC and LINE OUT. An I^{2}C engine programs the codec once after reset. Two slide switches select the bypass mode and freeze the adaptation, a push button resets the system, and four GPIO test points bring out the sample strobe, the output strobe, the busy flag and the frame clock for timing measurement. The complete design is written in plain Verilog, without any processor or vendor IP.

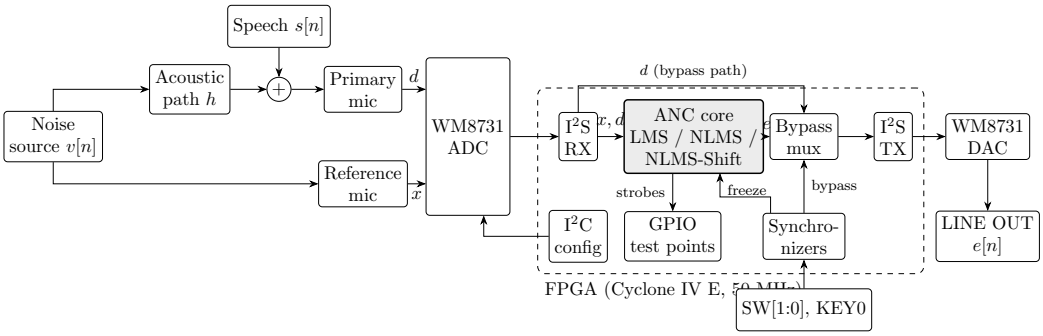


Figure 1: Block diagram of the ULLAP system. The acoustic part on the left is simulated in the experiments (Section 7), and all the blocks from the codec onwards run on the DE2-115 board.

4. Algorithms and fixed-point model

4.1. Signal model and cost function

The reference microphone receives only the noise, whereas the primary microphone receives the speech, the noise filtered by the acoustic path, and

a small sensor noise n_s. The two inputs are expressed as,

$$x[n] = v[n], \tag{1}$$

$$d[n] = s[n] + \sum_{j=0}^{M-1} h_j \, v[n-j] + n_s[n]. \tag{2}$$

Let the last L reference samples be collected in $\mathbf{x}[n] = [x[n], \ldots, x[n-L+1]]^T$ and the weights in $\mathbf{w}[n] = [w_0[n], \ldots, w_{L-1}[n]]^T$. The filter estimates the noise component of $d[n]$, and the remaining part forms the system output, which is given by,

$$y[n] = \mathbf{w}^T[n] \, \mathbf{x}[n] = \sum_{i=0}^{L-1} w_i[n] \, x[n-i], \tag{3}$$

$$e[n] = d[n] - y[n]. \tag{4}$$

Since s is uncorrelated with v, minimizing the mean-square error, which is formulated as,

$$J(\mathbf{w}) = E\{e^2[n]\} = E\{d^2[n]\} - 2\,\mathbf{w}^T\mathbf{p} + \mathbf{w}^T\mathbf{R}\,\mathbf{w}, \tag{5}$$

$$\mathbf{R} = E\{\mathbf{x}[n]\,\mathbf{x}^T[n]\}, \qquad \mathbf{p} = E\{d[n]\,\mathbf{x}[n]\}, \tag{6}$$

removes the noise and leaves the speech unaffected. Here, $\mathbf{R}$ denotes the autocorrelation matrix of the reference vector and $\mathbf{p}$ denotes the cross-correlation vector between $d[n]$ and $\mathbf{x}[n]$. The minimum of this cost is the Wiener solution $\mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p}$, which equals the acoustic path when $L \geq M$ and n_s is white.

4.2. LMS, NLMS and NLMS-Shift

In LMS, the gradient $\nabla J = -2(\mathbf{p} - \mathbf{R}\mathbf{w})$ is replaced with its instantaneous estimate $-2e[n]\mathbf{x}[n]$. All the three algorithms use the resulting update, which is expressed as,

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu[n]\, e[n]\, \mathbf{x}[n]. \tag{7}$$

The three algorithms differ only in the step $\mu[n]$. With the window power $P[n] = \|\mathbf{x}[n]\|^2$, the step sizes of LMS, NLMS and NLMS-Shift are given by,

$$\mu_{\text{LMS}}[n] = \mu, \qquad 0 < \mu < \frac{2}{L\,\sigma_x^2}, \tag{8}$$

$$\mu_{\text{NLMS}}[n] = \frac{\tilde{\mu}}{P[n] + \epsilon}, \qquad 0 < \tilde{\mu} < 2, \tag{9}$$

$$\mu_{\text{Shift}}[n] = 2^{-(k[n]+K)}, \tag{10}$$

$$k[n] = \lfloor \log_2(P[n] + \epsilon) \rfloor + 1, \tag{11}$$

where σ_x^2 denotes the reference variance and $\epsilon > 0$ represents a small regularizer. The LMS bound varies with the input level, and hence a fixed μ becomes either too slow for quiet inputs or unstable for loud inputs. In contrast, the NLMS bound does not vary. In the hardware, the power is updated with the exact recursion,

$$P[n] = P[n-1] + x^2[n] - x^2[n-L], \tag{12}$$

which needs two squaring operations per sample instead of L. In NLMS-Shift, the exponent $k[n]$ is simply the bit length of $P[n] + \epsilon$. This exponent is found by a leading-one detector, and the multiplication by $\mu[n]$ becomes an arithmetic right shift of $e[n]\,x[n-i]$ by $k[n] + K$ places.

Proposition 1. *For every n, the NLMS-Shift step* (10) *satisfies*

$$\frac{2^{-K}}{2\,(P[n] + \epsilon)} \;\le\; \mu_{\mathit{Shift}}[n] \;<\; \frac{2^{-K}}{P[n] + \epsilon}, \tag{13}$$

In other words, the NLMS-Shift step never falls more than a factor of two below the NLMS step with $\tilde{\mu} = 2^{-K}$, and it satisfies the NLMS stability bound $0 < \tilde{\mu} < 2$ for $K \ge 0$.

Proof. By (11), $2^{k[n]-1} \le P[n] + \epsilon < 2^{k[n]}$. Taking reciprocals and multiplying by 2^{-K} gives (13), since $\mu_{\text{Shift}}[n] = 2^{-K}\,2^{-k[n]}$. □

The exact relation between the two steps is expressed as,

$$\mu[n] = \frac{r[n]\,2^{-K}}{P[n] + \epsilon}, \qquad r[n] = \frac{P[n] + \epsilon}{2^{\lfloor \log_2(P[n]+\epsilon) \rfloor + 1}} \in [0.5, 1), \tag{14}$$

Hence, NLMS-Shift is NLMS with the time-varying step $2^{-K} r[n]$ (Section 8.4). When the input level is unknown over several octaves, the fractional part $u = \log_2(P+\epsilon) - \lfloor \log_2(P+\epsilon) \rfloor$ is spread uniformly over $[0, 1)$, and the mean ratio is given by,

$$E\{r\} = \int_0^1 2^{u-1}\, du = \frac{1}{2 \ln 2} \approx 0.72. \tag{15}$$

4.3. Fixed-point model

The signals and the weights are represented as signed $Q1.F$ numbers with $W = 12$ bits and $F = W - 1$ fraction bits. Using the rounding shift and the saturation functions defined as,

$$\mathrm{rs}(a, m) = \left\lfloor \frac{a + 2^{m-1}}{2^m} \right\rfloor, \tag{16}$$

$$\mathrm{sat}_B(a) = \min\big(\max\big(a, -2^{B-1}\big),\, 2^{B-1} - 1\big), \tag{17}$$

the integer datapath operates on the integer samples $x_i = x[n-i] \cdot 2^F$ and the weights w_i as follows,

$$a[n] = \mathrm{sat}_{B_a}\Big(\sum_{i=0}^{L-1} w_i\, x_i\Big), \tag{18}$$

$$B_a = 2W + \lceil \log_2 L \rceil, \tag{19}$$

$$y[n] = \mathrm{sat}_W\big(\mathrm{rs}(a[n], F)\big), \tag{20}$$

$$e[n] = \mathrm{sat}_W\big(d[n] - y[n]\big), \tag{21}$$

$$w_i \leftarrow \mathrm{sat}_W\big(w_i + \Delta w_i\big), \tag{22}$$

where the weight increments of LMS, NLMS and NLMS-Shift are given by,

$$\Delta w_i^{\mathrm{LMS}} = \mathrm{rs}\big((M_\mu\, e[n])\, x_i,\ F + 10\big), \tag{23}$$

$$\Delta w_i^{\mathrm{NLMS}} = \mathrm{rs}\big(g[n]\, x_i,\ F\big), \tag{24}$$

$$g[n] = \mathrm{sat}_{W+4}\left(\left\lfloor \frac{2N[n] + Q[n]}{2Q[n]} \right\rfloor\right), \tag{25}$$

$$\Delta w_i^{\mathrm{Shift}} = \mathrm{rs}\big(e[n]\, x_i,\ b(Q[n]) - F + K\big), \tag{26}$$

Here, $\mu = M_\mu\, 2^{-10}$, $N[n] = M_\mu\, e[n]\, 2^{2F-10}$, $Q[n] = P[n] + 2^{2F-E}$ for $\epsilon = 2^{-E}$, and $b(\cdot)$ denotes the bit length of a positive integer. Every point at which a value is narrowed saturates and sets a sticky flag, so that an overflow is never left unnoticed. The integer model (`py/fixedpoint.py`, `py/rtl_model.py`) uses exactly these widths and generates the golden vectors.

5. Hardware architecture

5.1. Core

The datapath of the core (`rtl/anc_core.v`, vendor-neutral Verilog) is shown in Fig. 2. The core accepts one sample pair per `in_valid` strobe and performs all its arithmetic on a single multiplier with a three-stage pipeline (operand registers, product register and consume stage). A finite-state machine steps through each sample. It spends two cycles on squaring the new and the dropped reference samples for (12) (NLMS and NLMS-Shift only), L cycles on the filter sum (18) and one cycle on y and e (21). Thereafter, the step computation is carried out, followed by L cycles of weight updates (22). NLMS adds a sequential restoring divider for the gain $g[n]$, whereas NLMS-Shift uses a leading-one detector and a shift-amount register in its place. A `freeze` input holds the weights while the filtering continues.

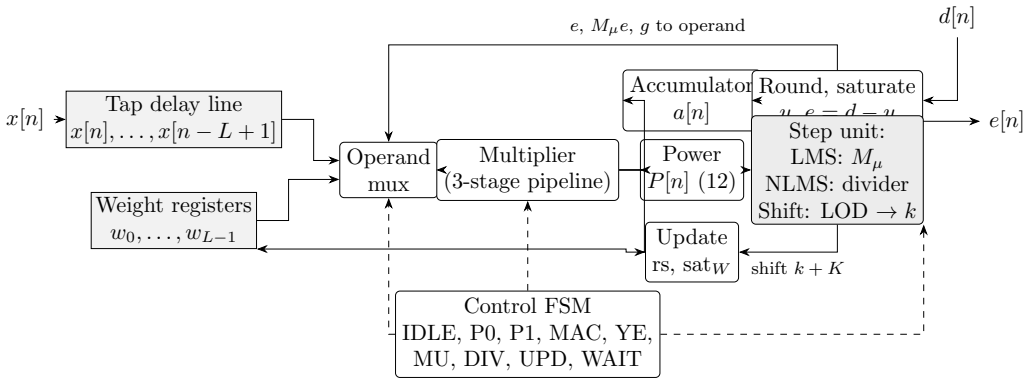


Figure 2: Datapath of the ANC core. A single multiplier is shared by the power update, the filter sum, the step computation and the weight updates, and only the step unit differs between the three algorithms. Dashed lines denote control signals.

This schedule fixes both the input-to-output latency and the number of busy cycles per sample, which are expressed as,

$$N_{\text{lat}} = \begin{cases} L + 3 & \text{LMS}, \\ L + 5 & \text{NLMS, NLMS-Shift}, \end{cases} \tag{27}$$

$$N_{\text{busy}} = \begin{cases} 2L + 9 & \text{LMS}, \\ 2L + 13 + N_{\text{div}} & \text{NLMS}, \\ 2L + 8 & \text{NLMS-Shift}, \end{cases} \tag{28}$$

The restoring divider performs one iteration per dividend bit, $N_{\mathrm{div}} = W + B_\mu + 2F + 4 + \lceil \log_2 L \rceil$, where B_μ denotes the signed width of M_μ, and hence the NLMS cycle count increases with the step-size constant. For $L = 32$ and the tuned settings, the above formulas give latencies of 35, 37 and 37 cycles and busy times of 73, 130 ($N_{\mathrm{div}} = 53$) and 72 cycles, which are the same values measured on the board (Section 8.6).

5.2. Board wrapper

The DE2-115 wrapper around the core contains an I^{2}C engine that configures the WM8731 at reset (11 registers, each write being acknowledge-checked), an I^{2}S receiver and transmitter, two-flop synchronizers on the reset and the switches, a frame-latched bypass multiplexer, an overrun flag, and four registered test points on the GPIO header. The I^{2}S blocks run on the 50 MHz system clock and sample the bit and word clocks of the codec as data. The codec acts as the clock master and is driven from a 12.5 MHz clock with its sample-rate register set to 1536 fs. Therefore, the sample rate and the number of clock cycles per sample are given by,

$$f_s = \frac{f_{\mathrm{MCLK}}}{1536} = \frac{12.5\ \mathrm{MHz}}{1536} = 8.138\ \mathrm{kHz}, \tag{29}$$

$$N_s = \frac{f_{\mathrm{clk}}}{f_s} = \frac{50\ \mathrm{MHz}}{8.138\ \mathrm{kHz}} = 6144, \tag{30}$$

where f_s is the rate used in the simulations and N_s is the number of clock cycles per sample. The fraction of each sample period during which the core is busy is expressed as,

$$U = \frac{N_{\mathrm{busy}}}{N_s} \tag{31}$$

From (28), this fraction is 1.19%, 2.12% and 1.17% for LMS, NLMS and NLMS-Shift, respectively. As mentioned earlier, no processor or vendor IP is involved.

6. Verification

The verification steps are listed in Table 1. The core testbench checks every $y[n]$ and $e[n]$, the final weights, the sticky flags and the latency against golden vectors for 68 configurations (W from 8 to 31, L from 2 to 32, saturation corner cases and freeze patterns). The top-level testbench adds a behavioral WM8731 model, which checks every I^{2}C write and follows the

programmed sample rate. This testbench runs the tests T1–T10, namely configuration, NACK handling, bypass, end-to-end cancellation against golden vectors, freeze, reset, flags, clocks, a 45-point skew sweep, the 8 kHz rate and the GPIO timing. To confirm that the testbenches are actually able to fail, faults were injected deliberately (mutation tests), and all of them were detected. The Verilator `-Wall` lint reports no unwaived warning.

Table 1: Verification summary. RTL suites pass in both Icarus Verilog and Questa (`results/metrics/iverilog_regression_summary.txt`, `results/local_run_da0b6f4/`).

Suite	Tests	Result
Core golden vectors (`tb_anc_core`)	68	all pass, bit-exact
Protocol corner cases	4	pass
Freeze hold/toggle variants	3	pass
Top level with codec model (T1–T10)	11	pass
Python unit tests	53	pass
Verilator lint (`-Wall`)	–	0 unwaived
On-board Signal Tap captures (T9)	25	all bit-exact
On-board freeze, DAC word, 10-min soak	4	pass

7. Experimental set-up

The synthetic scenarios follow the reference model $x = v$, $d = s + h * v + n_s$ at 8 kHz with 16,000 samples. Four scenarios are considered, namely stationary colored noise, amplitude steps ($\times 3$ followed by $\times 0.25$), an abrupt path switch at the midpoint, and a recorded chord as the noise source. The step sizes and the regularizers were selected on seeds 101–103 by maximizing the mean final noise reduction over the stationary, amplitude-step and path-switch scenarios, with no overflow allowed. This selection gave $\mu = 0.25$ for LMS; $\mu = 0.25$ and $\epsilon = 1$ for NLMS; and $K = 3$ and $\epsilon = 0.25$ for NLMS-Shift. All the reported results use seeds 1–20. The final noise reduction is computed over the last quarter $\mathcal{N}$ of the samples as,

$$\mathrm{NR} = 10 \log_{10} \frac{\sum_{n \in \mathcal{N}} \big(d[n] - s[n]\big)^2}{\sum_{n \in \mathcal{N}} \big(e[n] - s[n]\big)^2} \text{ dB}, \tag{32}$$

and each table entry gives the mean over the $S = 20$ seeds along with its 95% confidence interval, which is given by,

$$\overline{\mathrm{NR}} \pm t_{0.975,\,S-1}\,\frac{\sigma_{\mathrm{NR}}}{\sqrt{S}}, \qquad t_{0.975,19} = 2.093. \tag{33}$$

For comparing two algorithms A and B, the paired per-seed differences $\Delta_k = \mathrm{NR}_{\mathrm{A},k} - \mathrm{NR}_{\mathrm{B},k}$ are used. The two algorithms are considered equivalent when two one-sided tests (TOST) both reject, at $p < 0.05$, the hypothesis that the mean difference lies outside a margin $\pm\delta$ with $\delta = 0.5$ dB, where the two null hypotheses are expressed as,

$$H_{01}: \ \bar{\Delta} \le -\delta, \qquad H_{02}: \ \bar{\Delta} \ge \delta. \tag{34}$$

Use of generative AI. During this work, the authors used Claude (Anthropic) as an assistant to write and debug the analysis scripts and testbench code, to search for and summarize the literature, and to draft and edit the text of this manuscript. [TBD: Authors: add any other AI tools you used (for example for local runs or literature searches) and adjust the list above if needed.] The authors checked every reported number against the committed result files and board captures, verified each reference against its source, and take full responsibility for the content of this article.

For the real recordings, four noise files (air conditioner, vacuum cleaner, copy machine and babble) and ten utterances were taken from the public MS-SNSD corpus[1] and resampled to 8 kHz. In each seed, a noise segment, an utterance and a random 16-tap acoustic path are drawn at 0 dB input SNR, both with and without speech leaking into the reference at -20 dB. The noise reduction, the short-time objective intelligibility (STOI) measure and the narrow-band perceptual evaluation of speech quality (PESQ, ITU-T Recommendation P.862) are computed after a 2 s convergence window.

8. Results

8.1. Synthetic scenarios

Table 2 shows that NLMS-Shift ties with NLMS on stationary noise and amplitude steps, and ties with LMS on stationary noise and the path switch. On amplitude steps, both normalized algorithms outperform LMS by about

[1] `https://github.com/microsoft/MS-SNSD`

Table 2: Final noise reduction, dB, mean [95% CI] over seeds 1–20, $W = 12$, $L = 32$ (`results/paper_numbers.csv`).

Scenario	LMS	NLMS	NLMS-Shift
Stationary	22.39 [22.12, 22.66]	22.28 [22.01, 22.54]	22.37 [22.11, 22.63]
Amplitude steps	22.36 [21.72, 23.00]	24.16 [23.58, 24.73]	24.33 [23.88, 24.78]
Path switch	18.95 [18.81, 19.09]	18.53 [18.39, 18.67]	18.91 [18.76, 19.06]
Recorded chord	5.69 [5.66, 5.73]	5.73 [5.69, 5.76]	5.46 [5.42, 5.50]

2 dB. In this scenario, the fixed LMS step is either too slow after the drop or too noisy before it: no LMS step size reaches 22 dB on stationary noise and 24 dB on amplitude steps at the same time, whereas both NLMS variants reach both values (Fig. 3). On the recorded chord, NLMS-Shift loses 0.2–0.3 dB. The paired per-seed tests in Table 3 quantify the term "tie". Every difference between NLMS-Shift and NLMS falls inside a ±0.5 dB equivalence margin (TOST, $p < 0.05$), even though some differences are statistically non-zero, such as +0.38 dB on the path switch. The 1.8–2.0 dB advantage of the normalized algorithms over LMS on amplitude steps is significant ($p < 10^{-7}$).

Table 3: Paired differences in final NR over seeds 1–20 (`results/metrics/paired_stats.csv`).

Scenario	Pair	Mean diff. (dB)	95% CI	p (paired t)	TOST ±0.5 dB
Stationary	NLMS-Shift − NLMS	+0.10	[+0.01, +0.18]	0.028	equivalent
Stationary	NLMS-Shift − LMS	-0.02	[-0.07, +0.04]	0.58	equivalent
Stationary	NLMS − LMS	-0.11	[-0.18, -0.04]	0.0038	equivalent
Amplitude steps	NLMS-Shift − NLMS	+0.17	[-0.12, +0.47]	0.23	equivalent
Amplitude steps	NLMS-Shift − LMS	+1.97	[+1.51, +2.43]	2.8e-08	not equiv.
Amplitude steps	NLMS − LMS	+1.79	[+1.38, +2.21]	2.5e-08	not equiv.
Path switch	NLMS-Shift − NLMS	+0.38	[+0.30, +0.47]	1.9e-08	equivalent
Path switch	NLMS-Shift − LMS	-0.04	[-0.07, -0.01]	0.018	equivalent
Path switch	NLMS − LMS	-0.42	[-0.50, -0.34]	1.1e-09	equivalent
Recorded chord	NLMS-Shift − NLMS	-0.27	[-0.30, -0.24]	7.7e-14	equivalent
Recorded chord	NLMS-Shift − LMS	-0.24	[-0.26, -0.22]	2.6e-16	equivalent
Recorded chord	NLMS − LMS	+0.03	[+0.00, +0.06]	0.036	equivalent

Figs. 4–6 show single runs (seed 1) at the fair-tuned settings. Fig. 4 illustrates the running noise reduction in each scenario, Fig. 5 shows the residual error power against the input noise power for stationary noise, and Fig. 6 depicts the response to the amplitude steps. After the drop at 1.2 s, LMS settles about 3 dB below both normalized algorithms, which agrees with Table 2.

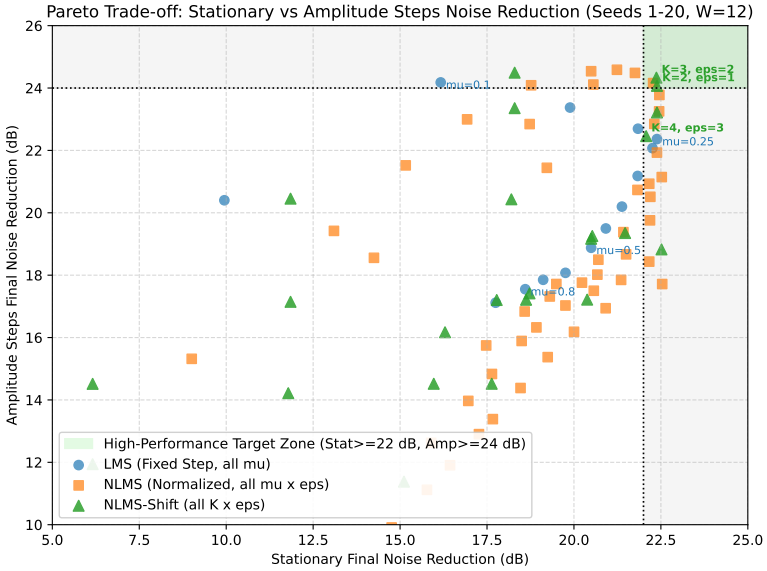


Figure 3: Stationary versus amplitude-step noise reduction across step sizes (seeds 1–20).

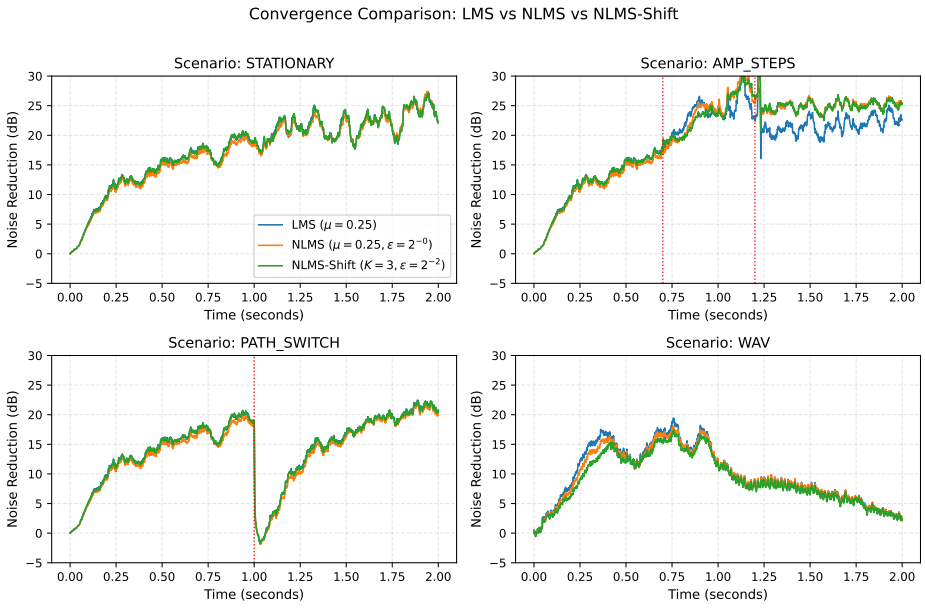


Figure 4: Running noise reduction in the four synthetic scenarios (seed 1, $W = 12$, $L = 32$, fair-tuned settings). Dotted lines mark the amplitude steps and the path switch.

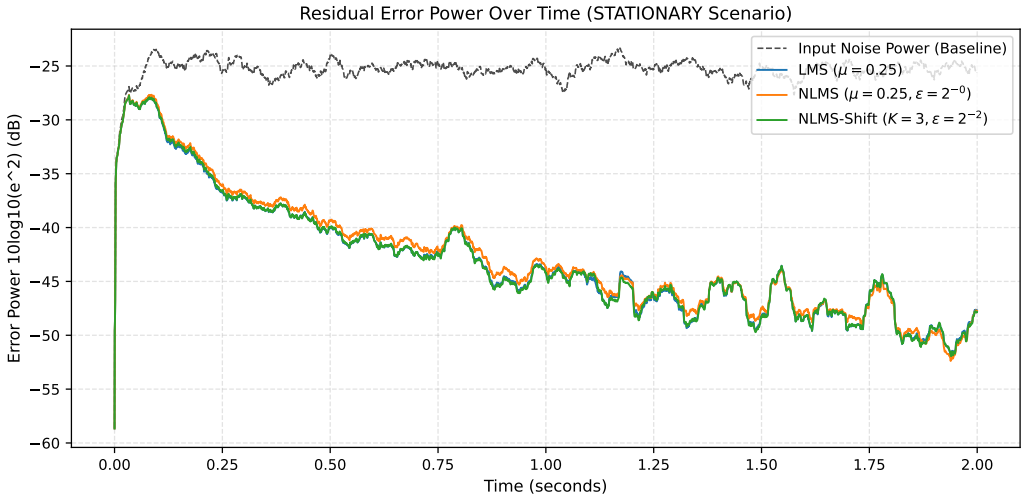


Figure 5: Residual error power over time against the input noise power, stationary scenario (seed 1).

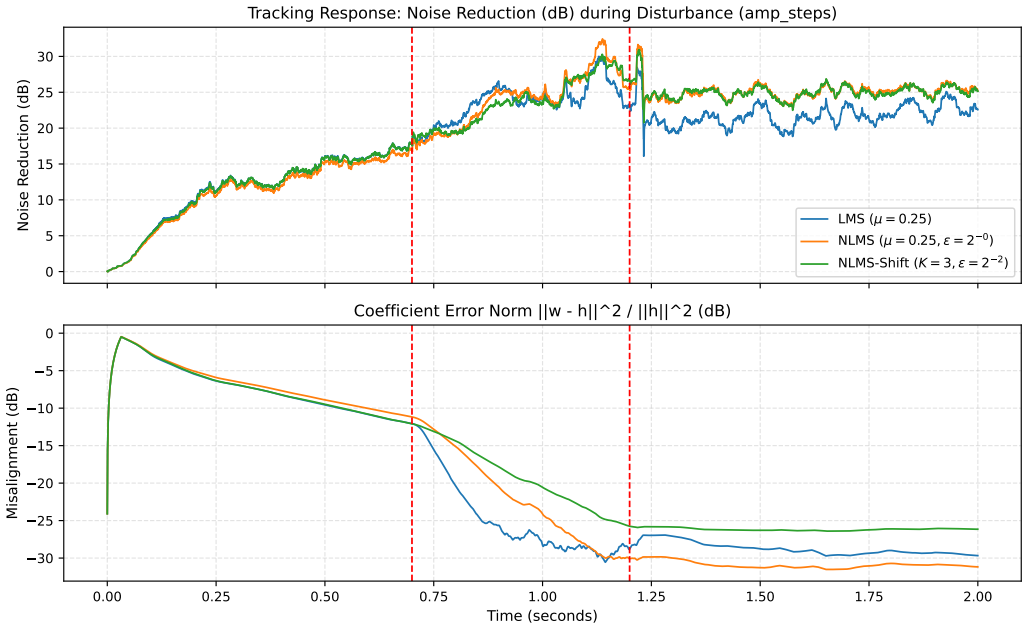


Figure 6: Tracking of the amplitude-step disturbance (seed 1): noise reduction and coefficient misalignment.

8.2. Real noise and speech

Table 4: Real recorded noise and speech (MS-SNSD), fixed point $W = 12$, mean ± 95% CI over 20 seeds, no speech leakage into the reference; STOI and PESQ of e against s, with the unprocessed input in brackets (`results/metrics/real_audio_summary.csv`).

Noise	Metric	LMS	NLMS	NLMS-Shift
Air conditioner	NR (dB)	11.65 ± 0.75	12.04 ± 0.82	11.89 ± 0.80
	STOI (input 0.784)	0.948	0.947	0.948
	PESQ (input 1.54)	2.34	2.30	2.33
Vacuum cleaner	NR (dB)	14.32 ± 0.32	14.90 ± 0.33	14.56 ± 0.35
	STOI (input 0.730)	0.982	0.983	0.982
	PESQ (input 1.39)	2.81	2.81	2.81
Copy machine	NR (dB)	14.02 ± 0.38	14.63 ± 0.34	14.39 ± 0.32
	STOI (input 0.694)	0.977	0.978	0.977
	PESQ (input 1.36)	2.74	2.75	2.73
Babble	NR (dB)	11.95 ± 0.42	12.43 ± 0.49	12.27 ± 0.46
	STOI (input 0.719)	0.956	0.955	0.955
	PESQ (input 1.48)	2.48	2.47	2.47

For all four real noises (Table 4), the filter roughly doubles the PESQ and brings the STOI close to 1. The ranking of the algorithms remains the same in every case: NLMS leads LMS by 0.4–0.6 dB, and NLMS-Shift trails NLMS by 0.15–0.35 dB. The noise reduction is lower than that in Table 2, but this is not because the real noise is harder to predict. When the speech was removed from the primary input, the same filters on the same noise segments and paths achieved 18.5–19.2 dB on the air conditioner and 22.4–31.9 dB on the other three noises, which is equal to or higher than the reduction on synthetic stationary noise (`py/exp_real_audio_nospeech.py`, `results/metrics/real_audio_nospeech.csv`). Although the speech is uncorrelated with the reference, it still enters the error $e[n]$ that drives the weight update, and it costs 6.4–17.9 dB. Without speech, LMS is ahead of both normalized variants on all four noises, whereas with speech it loses the most (7.6–17.9 dB against 6.4–14.8 dB for NLMS), and hence the order is reversed. The usual remedy is to slow down or stop the adaptation while speech is present. The `freeze` input of the core (Section 5.1) could be driven by a voice-activity detector for this purpose, but this has not been evaluated in this work. When −20 dB of the speech leaks into the reference, the filter also cancels a part

of the speech. In this case, the noise reduction drops by 2.3–4.6 dB and the PESQ by 0.25–0.6, and saturation events appear (18,500 over 19.2 million processed samples, mostly for the air conditioner). Therefore, the position of the reference microphone matters more than the choice of the algorithm (`paper/tab_real_audio_xt.tex`).

8.3. Word length

At the tuned settings, the worst-scenario loss against a floating-point run of the same algorithm is 4.1–5.9 dB at $W = 10$, 0.9–1.2 dB at $W = 11$ and 0.3–0.4 dB at $W = 12$ (Fig. 7). Eleven bits is the shortest word length that stays within the 3 dB criterion, and 12 bits were retained for margin. The tonal chord recording shows an unusual behavior, in which the fixed-point filter outperforms floating point by up to 3.4 dB at $W = 11$ and loses 1.3 dB at $W = 13$. On such a narrow-band signal, the quantization changes the effective adaptation, and hence the chord result varies with the word length and should not be used to select it.

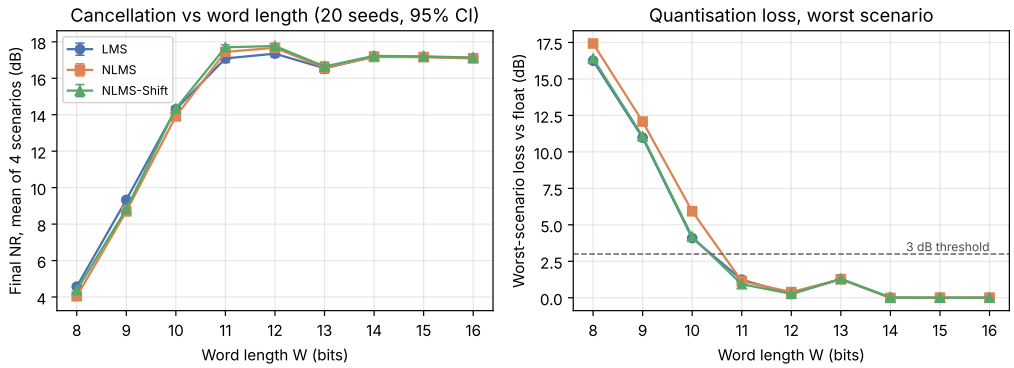


Figure 7: Noise reduction and worst-scenario quantization loss versus word length (20 seeds).

8.4. Equivalent step size of NLMS-Shift

From (14), NLMS-Shift and NLMS differ only in the factor $r[n]$. When averaged over input levels spread uniformly on a logarithmic scale, $E[r] = \int_0^1 2^{u-1}\,du = 1/(2\ln 2) \approx 0.72$. At any single level, however, r depends on the position of $P + \epsilon$ inside its octave: it increases with the level and halves when $P + \epsilon$ crosses a power of two. With $L = 32$ and $\epsilon = 2^{-2}$, the crossing

$P+\epsilon = 1/2$ occurs at a reference RMS of -21.1 dBFS, which is 3 dB above the level used in the simulations. To examine this effect, the input level was swept over 12 dB in 0.75 dB steps in the integer model (`py/exp_equiv_step.py`), on the stationary scenario (seeds 1–5) and on a two-tone reference (Fig. 8). On broadband noise, the window power varies enough to smooth out the sawtooth, and the mean ratio $\bar{r}$ changes gradually between 0.56 and 0.78. On the tonal reference, P is almost constant, and $\bar{r}$ drops from 0.87 to 0.62 within 0.75 dB of level. In both cases, NLMS with the constant step $\bar{r}\,2^{-K}$ reproduces NLMS-Shift. The final noise reduction agrees within 0.27 dB at all 17 broadband levels, and within 0.55 dB at every tonal level except the one that straddles the crossing (1.7 dB), while the convergence time agrees within 8%. In contrast, NLMS with the nominal step 2^{-K} deviates by up to 3.5 dB and 48%.

Hence, the shift changes nothing in the adaptation except the step size. It converts the fixed NLMS step into a step between 0.5 and 1 times the nominal step, which is set by the input level. On broadband input, this is a smooth change of gain, and selecting K (and ϵ) for the expected level absorbs it. On tonal input, the step changes by up to a factor of about 0.7 when the level crosses a power-of-two boundary of $P + \epsilon$, which never happens in a fixed-step NLMS.

8.5. Hardware cost

Table 5: Complete board design (core, codec interface, synchronizers, status logic) on Cyclone IV E EP4CE115F29C7, $W = 12$, $L = 32$, Quartus Prime Lite 25.1, commit da0b6f4 (`results/quartus_reports/board_da0b6f4/`, `results/metrics/power_activity.csv`).

	LMS	NLMS	NLMS-Shift
Logic elements	1,792	2,626	2,610
Registers	1,123	1,404	1,186
9-bit multipliers	4	4	4
$F_{\max}$, slow 85 °C (MHz)	70.08	59.75	58.66
Setup slack at 50 MHz, slow 85 °C (ns)	5.731	3.263	2.953
Latency (cycles)	35	37	37
Busy cycles per sample (measured on the board)	73	130	72
RTL bit toggles per sample	5,668	7,146	6,012
Core dynamic power, vectorless (mW)	20.79	24.36	23.77

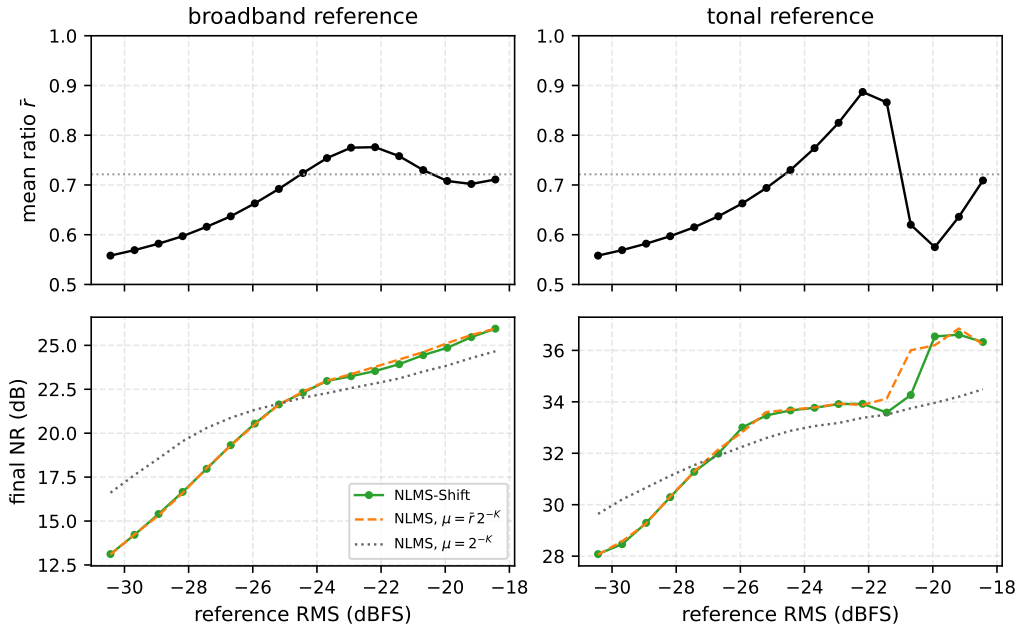


Figure 8: Equivalent step of NLMS-Shift versus reference level (integer model, $W = 12$, $L = 32$, $K = 3$, $\epsilon = 2^{-2}$). Top: mean ratio $\bar{r}$ of (14); dotted: $1/(2 \ln 2)$. Bottom: final noise reduction of NLMS-Shift, of NLMS with step $\bar{r}\,2^{-K}$ and of NLMS with step 2^{-K}. Left: stationary scenario, seeds 1–5; right: two-tone reference (`results/metrics/equiv_step.csv`).

LMS is about 31% smaller than either of the normalized variants. NLMS-Shift and NLMS require almost the same logic (2,610 and 2,626 logic elements), since the area saved on the divider is consumed by the leading-one detector, the shifter and the wider update path. However, NLMS-Shift has 15% fewer registers, needs 45% fewer busy cycles per sample and toggles 16% fewer bits per sample. All three designs meet the 50 MHz constraint with a setup slack of at least +2.953 ns and a hold slack of +0.401 ns, without any critical warnings. At 8.138 kHz, even the busiest variant uses only 2% of the 6,144-cycle sample period, and hence a single core could serve many channels or much longer filters.

The power values are Quartus vectorless estimates with the default toggle rate of 12.5%, which Quartus itself rates as low confidence. The core static power (about 98.7 mW) dominates the total power of 169 to 173 mW. The dynamic estimates rank the designs as LMS < NLMS-Shift < NLMS, which

is the same order as the toggle counts. However, a fixed toggle rate mostly tracks the area and ignores the fact that the datapath remains idle for 98% of each sample period, and hence these values are upper bounds and not the energy per sample. A gate-level, activity-annotated estimate is left for future work, because the post-fit netlist flattens the hierarchy observed by the top-level testbench, and a netlist-only testbench is therefore required. A fair comparison with Huang and Yao [4], who used the same board, would report the energy per sample as the dynamic power divided by the sample rate from an activity-annotated Power Analyzer run. Since this result is not yet available, no energy comparison with their design is claimed. Moreover, their design is fully parallel and runs at megasamples per second, whereas ULLAP shares one multiplier and runs at 8.138 kHz, so the area and energy have to be compared per tap and per sample rather than as raw totals.

8.6. On-board results

The board evidence was recorded with Signal Tap on the FPGA itself, and hence no external recorder or oscilloscope was required. One logic-analyzer instance stores, once per sample, the two codec samples entering the core (x and d, register outputs `x_reg` and `d_reg`) and the core output e. This instance uses a transitional storage qualifier on `out_valid`, triggers when the reset button is released, and holds 32,768 samples (4.0 s). A second instance samples the four GPIO test points at 50 MHz. The stimuli were the noise-only versions of the four scenarios, which were played from a laptop into LINE IN after a 1 kHz calibration tone. The reset was pressed during the tone, so that every capture starts from zero weights on real codec data at about -25 dBFS, and no sample in any capture reaches full scale. The analysis script and the raw exports are available in the repository (`py/analyze_board_capture.py`, `board_evidence/`).

Bit-exactness. For each of the 12 captures (three algorithms and four scenarios), the integer model of Section 4 was started from zero state at the first sample after the reset and was fed with the captured x and d. The model reproduced every captured $e[n]$, i.e., 351,723 samples in total and 28,672 to 32,768 (3.5 to 4.0 s) per capture, without a single mismatch. Therefore, the fixed-point arithmetic on the chip is exactly the arithmetic evaluated in simulation, even with an analog input that carries the noise and offsets of the codec.

Timing. The GPIO capture shows an `in_valid` period of 6,144 clock cycles (8,138.02 Hz), an `in_valid` to `out_valid` spacing of 38, 36 and 38

cycles for NLMS-Shift, LMS and NLMS (the core latency plus one accept cycle, i.e., 0.76, 0.72 and 0.76 μs), and busy times of 72, 73 and 130 cycles. All these values are equal to those of the RTL simulation with the same parameters. The builds with Signal Tap still meet the 50 MHz constraint (setup slack of +2.65, +5.68 and +3.78 ns), while Table 5 refers to the builds without the logic analyzer.

Fig. 10 shows the three algorithms on the same amplitude-step stimulus, aligned to the start of the scenario. (Since KEY0 was pressed by hand during the calibration tone, the reset occurs at a slightly different point of the stimulus in each capture.) All three algorithms settle to about 15 dB in the first 1.5 s. At 2.1 s, the level of d rises by 7.4 dB, and the algorithms follow the step with only a short burst in e. The residual power after the step (−47 to −52 dBFS) is lower than that before the step (−44 dBFS), and hence the noise reduction rises to 25 to 30 dB. Over the last quarter of the window, the reduction is about 40 dB below 1.5 kHz and 11 to 15 dB near the band edge (3.5 to 3.9 kHz).

Noise reduction on the simulation signal. In a second session, the 2 s, seed-1 noise-only signal of each scenario, i.e., exactly the 16,000-sample signal used in the simulations, was played once after the calibration tone, so that the complete scenario fits inside the capture. Its start was located by correlating the captured x with the stimulus (correlation of 0.95 to 0.99), and $\mathrm{NR}_{\mathrm{final}}$ was computed over the same last 25 % as in the simulation. All 12 captures (28,672 samples after the reset each) are again bit-exact. Table 6 compares the board with the integer model on the same signal. The board results are 2.9 to 10.6 dB below the simulation, by similar amounts for all three algorithms. The cause of this difference is the input level and not the hardware. The codec delivered x and d at about −27 dBFS, which is 3 dB below the simulation level, and at the tuned step sizes a lower input power slows down the adaptation within the 2 s. When the model is run at the level actually received by the board, it matches the board within 0.2 dB on stationary and path-switch noise (Fig. 9). The amplitude-step and recorded-noise scenarios end with a quiet final quarter (d at about −41 and −57 dBFS), and in these cases the board stays 1.7 to 8.1 dB below the level-matched model. At these levels, two analog effects are no longer small compared with d: the noise of the playback and codec path (−70 dBFS, measured in the silence after the stimulus) and an ADC offset that decays during the first second after the reset. For amplitude steps, even the best 32-tap least-squares filter fitted to the board samples of that window reaches only 24 dB, against 28 dB

on the ideal signal. On the board, the three algorithms remain within 0.7 dB of one another in every scenario.

Table 6: On-board NR_{final} (dB) on the 2 s seed-1 stimulus against the integer model on the same signal, at the simulation level and scaled to the level the board received (`results/board/session2/`). Every board capture is bit-exact.

Algorithm	Scenario	Board	Model, sim. level	Model, board level
LMS	stationary	15.88	22.77	15.76
	amplitude steps	17.13	27.71	25.23
	path switch	14.67	19.84	14.47
	recorded noise	5.99	9.11	7.78
NLMS	stationary	15.65	21.60	15.53
	amplitude steps	16.76	27.19	23.64
	path switch	14.03	19.06	14.04
	recorded noise	5.80	8.72	7.45
NLMS-Shift	stationary	15.80	22.68	15.78
	amplitude steps	16.57	25.79	22.10
	path switch	14.40	19.86	14.44
	recorded noise	5.46	8.63	7.32

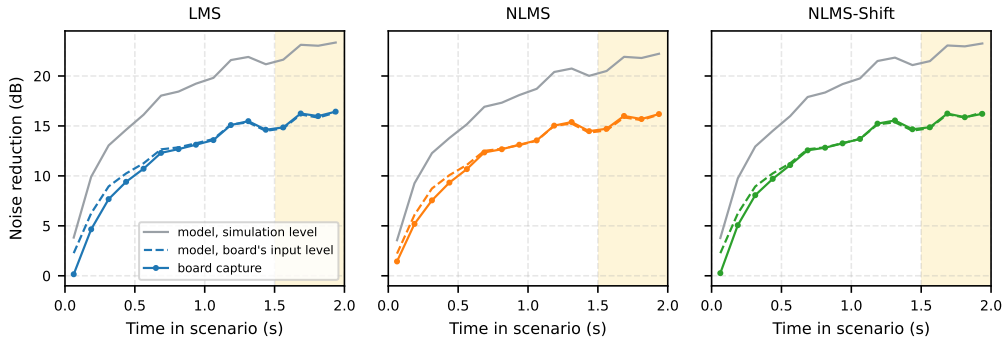


Figure 9: Running noise reduction (0.125 s windows) on the 2 s stationary stimulus: board capture, integer model at the simulation level, and integer model with the stimulus scaled to the level the board received. Shaded: the NR_{final} window.

Freeze, DAC word and endurance. With NLMS-Shift running on stationary noise, SW[1] was flipped by hand 1.44 s into the scenario. The captured

output is equal to the model with the adaptation stopped from that sample onwards (28,672 samples, no mismatch). From the switch to the end of the scenario, the noise reduction holds at 15.2 dB, against 15.4 dB just before the switch and 15.9 dB if the adaptation had continued. A third Signal Tap instance recorded the word sent to the DAC. In ANC mode, this word equals $\{e, 0000\}$, and in bypass mode (SW[0] = 1) its upper 12 bits equal d, in all 1,792 samples of each capture. The LINE OUT signal could not be recorded with the available equipment, and hence this check covers the chain only up to the DAC interface. After 10 minutes of continuous operation on looped noise, the GPIO timing remained unchanged (6,144 cycles per sample, 38 cycles from `in_valid` to `out_valid` and 72 busy cycles), and no saturation, NACK or overrun flag was set. The build with all three logic-analyzer instances still met timing (slack of +2.93 ns).

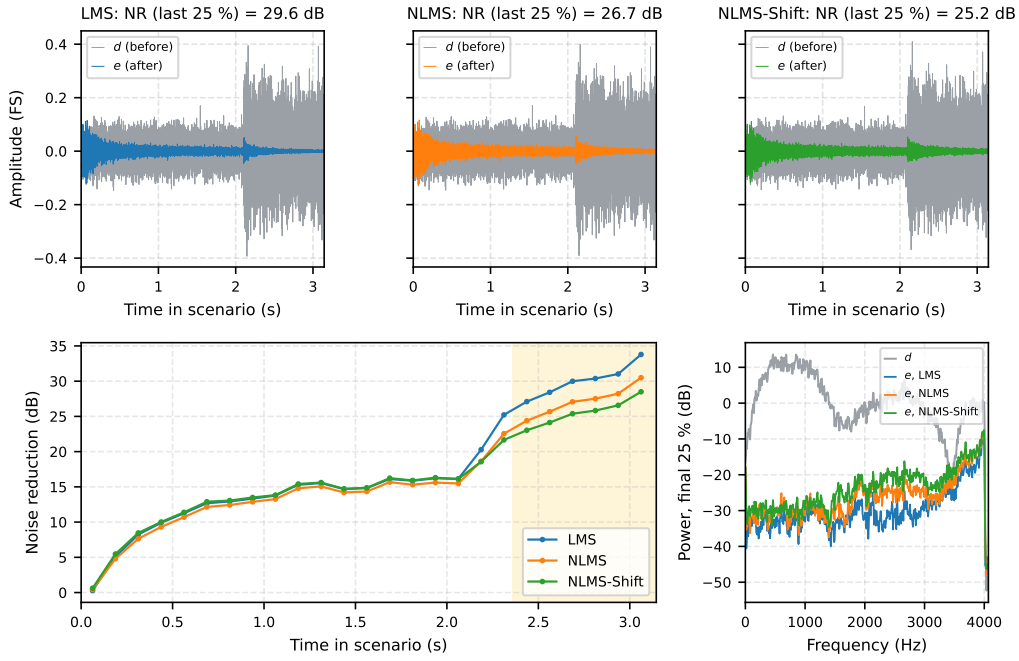


Figure 10: On-board Signal Tap captures of LMS, NLMS and NLMS-Shift on the same amplitude-step noise, from the start of the scenario: primary input d and output e (top), running noise reduction in 0.125 s windows (bottom left; shaded: last 25 % of the common window) and power spectra over that window (bottom right). Every output sample of all three captures equals the integer model's.

9. Discussion and limitations

The comparison in this work supports a narrow conclusion. The divider-free normalization retains the level independence of NLMS at roughly the cycle count of LMS. Therefore, it is the better choice when the noise level changes and when throughput or energy per sample is important, whereas LMS remains the smallest design when the level is steady. The comparison does not show that NLMS-Shift cancels better than NLMS, and Section 8.4 explains the reason. The shift changes only the step size, by a level-dependent factor between 0.5 and 1, and hence selecting K and ϵ for the expected input level matters more than the shift itself. In addition, tonal noise near a power-of-two boundary of $P+\epsilon$ experiences a step jump that is absent in NLMS. On real recordings, the speech at the primary input limits all three algorithms much more than the choice of the algorithm does (6.4–17.9 dB, against at most 0.6 dB between the algorithms).

The present study has certain limitations. The acoustic paths are simulated, including those used for the real recordings. Acoustic feedback (closed-loop howling) is not covered, and reference-based ANC is not a suitable structure for this problem in any case. The power is only a switching-activity proxy until a gate-level analysis is carried out. On the board, the output is shown to be equal to the model up to the DAC word, but the analog LINE OUT signal was not recorded. Finally, the absolute noise reduction depends on the input level. With the stimulus 3 dB below the simulation level, the board achieved 14 to 17 dB instead of 19 to 28 dB on the synthetic scenarios, and on stationary and path-switch noise the model reproduces this result at that level. Therefore, in a practical deployment, the codec input gain should be set such that the reference uses most of the 12-bit range, or the step size should be retuned for the expected level.

10. Conclusion

In this research, ULLAP is presented, which runs LMS, NLMS and NLMS-Shift on a single serial 12-bit, 32-tap core on a low-cost Cyclone IV board and compares the three algorithms with equal tuning effort. NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise, while needing 72 instead of 130 busy cycles per sample. All three algorithms remove 11.7 to 14.9 dB of real recorded noise, and 18.5 to 31.9 dB when the speech is removed from the primary input, which shows

that the speech, and not the noise, sets the limit. Moreover, NLMS-Shift is shown to be equivalent to NLMS with a level-dependent step between 0.5 and 1 times the nominal step. On the board, 25 Signal Tap captures of real codec data (724,459 samples) are bit-exact with the integer model, and the same holds for the freeze behavior and the DAC word. On stationary and path-switch noise, the measured noise reduction is equal to that of the model, at the input level received by the board, within 0.2 dB.

Declarations

Funding. [TBD: Authors: funding sources, or "No funding was received for this work."]
Competing interests. [TBD: Authors: competing interests, or "The authors declare no competing interests."]
Author contributions. [TBD: Authors: who did what (free text or CRediT).]
Data and code availability. All the RTL, testbenches, scripts and result files are available in the project repository [TBD: public URL and Zenodo DOI of the tagged release]. The real-audio study fetches its MS-SNSD files from a pinned commit and verifies them by SHA-256.
Use of generative AI. Documented in the experimental set-up (Section 7).

References

[1] L. Lu, K.-L. Yin, R. C. de Lamare, Z. Zheng, Y. Yu, X. Yang, B. Chen, A survey on active noise control in the past decade–Part I: Linear systems, Signal Processing 183 (2021) 108039. doi:10.1016/j.sigpro.2021.108039.

[2] V. Guidotti, G. Paim, L. M. G. Rocha, E. Costa, S. Almeida, S. Bampi, Power-efficient approximate Newton-Raphson integer divider applied to NLMS adaptive filter for high-quality interference cancelling, Circuits, Systems, and Signal Processing 39 (2020) 5729–5757.

[3] C.-Y. Yao, Y.-Z. Huang, The design of the NLMS adaptive filters using the fast-division approximation with CSD encoded divisors, IEEE Transactions on Circuits and Systems II: Express Briefs 71 (4) (2024) 2459–2463.

[4] Y.-Z. Huang, C.-Y. Yao, Enhancing the power efficiency of the NLMS adaptive IIR filter with fast binary reciprocal approximation, Circuits, Systems, and Signal Processing 45 (2026) 7561–7584. doi:10.1007/s00034-026-03536-z.

[5] T. Münzel, M. Kuntic, M. Molitor, M. Sørensen, A. Daiber, Transportation noise pollution as a cardiovascular risk factor: from epidemiological evidence to mechanistic insights, EXCLI Journal 24 (2025) 1713–1739. doi:10.17179/excli2025-9050.

[6] O. Hahad, M. Kuntic, S. Al-Kindi, I. Kuntic, D. Gilan, K. Petrowski, A. Daiber, T. Münzel, Noise and mental health: evidence, mechanisms, and consequences, Journal of Exposure Science and Environmental Epidemiology 35 (1) (2025) 16–23. doi:10.1038/s41370-024-00642-5.

[7] S. Chadha, K. Kamenov, A. Cieza, The world report on hearing, 2021, Bulletin of the World Health Organization 99 (4) (2021) 242–242A. doi:10.2471/BLT.21.285643.

[8] B. Yang, J. Yin, Z. Ye, S. Yang, L. Wang, Development and testing of an active noise control system for urban road traffic noise, Applied Sciences 14 (1) (2024) 175. doi:10.3390/app14010175.

[9] T. Jiang, J. Liu, C. Peng, S. Wang, Laboratory test of a vehicle active noise-control system based on an adaptive step size algorithm, Applied Sciences 13 (1) (2023) 225. doi:10.3390/app13010225.

[10] L. Zhu, X. Zhong, H. Wang, X. Qiu, S. Wu, D. Mao, An objective and subjective study on global active control of engine noise in a car, Journal of Vibration and Control 30 (9–10) (2024) 2311–2321. doi:10.1177/10775463231174730.

[11] W. Zhu, Z. Gu, X. Chen, P. Xie, L. Luo, Z. Bai, A new hybrid adaptive self-loading filter and GRU-net for active noise control in a right-angle bending pipe of an air conditioner, Sensors 25 (20) (2025) 6293. doi:10.3390/s25206293.

[12] Y. Yin, F. Chen, Acoustic feedback cancellation algorithm for hearing aids based on a weighted error adaptive filter, Electronics 12 (7) (2023) 1528. doi:10.3390/electronics12071528.

[13] H. Husstedt, W. Hilgerdenaar, M. Frenz, F. Denk, J. Tchorz, Evaluation of impulse noise reduction in hearing aids with technical measurements and ratings of discomfort, Acta Acustica 7 (2023) 47. doi:10.1051/aacus/2023042.

[14] P. D. Tetteh, R.-M. O. M. Gyening, J. Arthur, J. A. Owusu, B. A. Kponyo, E. Ahene, J. J. Kponyo, Speech enhancement techniques for hearing aids: a systematic review, Universal Access in the Information Society 25 (3) (2026) 75. doi:10.1007/s10209-026-01342-7.

[15] L. Lu, K.-L. Yin, R. C. de Lamare, Z. Zheng, Y. Yu, X. Yang, B. Chen, A survey on active noise control in the past decade–Part II: Nonlinear systems, Signal Processing 181 (2021) 107929. doi:10.1016/j.sigpro.2020.107929.

[16] F. B. Félix, M. de Castro Magalhães, G. de Souza Papini, Improved active noise control algorithm based on the convex combination method, Journal of the Brazilian Society of Mechanical Sciences and Engineering 43 (2021) 163. doi:10.1007/s40430-021-02866-0.

[17] F. Li, H. Ding, Z. Wu, F. Qian, T. Liu, A. Iqbal, An improved stability compensation for feedforward active noise control systems with acoustic feedback, Journal of Mechanical Science and Technology 38 (2024) 507–518. doi:10.1007/s12206-024-0101-5.

[18] H. H. Thannoon, I. A. Hashim, Hardware implementation of a high-speed adaptive filter using a combination of systolic and convex architectures, Circuits, Systems, and Signal Processing 43 (2024) 1773–1791. doi:10.1007/s00034-023-02539-4.

[19] M. Jagadeeswari, P. Rishabavarthani, S. Kumarganesh, S. Kannadhasan, Enhanced adaptive filtering in audio systems: normalized least mean square with cut-set retiming, Analog Integrated Circuits and Signal Processing 126 (1) (2026) 12. doi:10.1007/s10470-025-02537-3.

[20] E. Karakaya, O. Aytar, FPGA implementation of NLMS and VSLMS algorithms for underwater acoustics applications, IEEE Access 14 (2026) 53860–53872. doi:10.1109/ACCESS.2026.3681282.

[21] S. Park, D. Park, Low-power FPGA realization of lightweight active noise cancellation with CNN noise classification, Electronics 12 (11) (2023) 2511. doi:10.3390/electronics12112511.

[22] F. Abdi, W. Cichalewski, W. Jałmużna, Ł. Butkowski, J. Branlard, A. Bellandi, G. Jabłoński, FPGA-based implementation of an adaptive noise controller for continuous wave superconducting cavity, Electronics 13 (1) (2024) 155. doi:10.3390/electronics13010155.

[23] M. Nagabushanam, S. Chakrasali, S. L. Gangadharaiah, S. H. Patel, G. Ramaiah, R. Basak, An optimized VLSI implementation of the least mean square (LMS) adaptive filter architecture on the basis of distributed arithmetic approach, Journal of The Institution of Engineers (India): Series B 106 (3) (2025) 861–870. doi:10.1007/s40031-024-01028-9.

[24] M. M. A. da Rosa, P. Ü. da Costa, E. A. C. da Costa, S. J. M. Almeida, G. Paim, S. Bampi, Design of a low power and robust VLSI power line interference canceler with optimized arithmetic operators, Analog Integrated Circuits and Signal Processing 112 (2) (2022) 247–261. doi:10.1007/s10470-022-02050-x.

[25] L. F. d. S. Coelho, L. Lovisolo, M. P. Tcheou, Adaptive filtering with reduced computational complexity using SOPOT arithmetic, IEEE Transactions on Circuits and Systems I: Regular Papers 69 (2) (2022) 746–756.